

SQL: Structured Query Language

Sistemas de Informação 1

Content

- Objectives
 - Notation
 - SQL commands
 - Example database structure
 - Create and delete a database
 - Create and delete a table
 - Data Types
 - Modify tables
 - Create and delete indexes
 - SELECT
 - Select all rows and all columns
 - Select specific columns, all rows
 - DISTINCT
 - Calculated fields
 - Filtering rows (WHERE)
 - Comparison
 - Interval (BETWEEN / NOT BETWEEN)
 - Member (IN / NOT IN)
 - Patterns (% and _)
 - Patterns
 - NULL
 - Sorting (one column)
 - Sorting (several columns)
 - Aggregation functions
 - COUNT(*)
 - COUNT(DISTINCT)
 - COUNT and SUM
 - MIN, MAX, AVG
 - Grouping (GROUP BY)
 - Group restrictions (HAVING)
 - HAVING
 - Updating the database: INSERT
 - Updating the database: UPDATE
 - Updating the database: DELETE
 - Deleting foreign keys
 - Subqueries
 - Subqueries with comparison
-

Content (cont.)

- Subquery and aggregation function
 - Subqueries (IN)
 - ANY / SOME
 - ALL
 - Queries with multiple tables: simple JOIN
 - Sorting a JOIN
 - JOIN three tables
 - Group by multiple columns
 - INNER JOIN
 - LEFT OUTER JOIN
 - RIGHT OUTER JOIN
 - FULL OUTER JOIN
 - EXISTS
 - Combine table results
 - UNION
 - INTERSECT
 - EXCEPT
 - VIEW
 - Advantages and Disadvantages of VIEW
 - Horizontal VIEW
 - Vertical VIEW
 - VIEW with GROUP and JOIN
-

Objectives

A database interaction language should:

- Allow creating databases and relationship structures
- Allow performing tasks on the data (e.g., inserting, modifying, and deleting) Performing simple and complex queries
- Allow these tasks to be performed with minimal effort for the user
- Have an easy to learn command structure
- Be portable: it should be possible to use the same command in different DBMSs

SQL is one of these languages.

Notation

- Capital letters are used for reserved words. (Ex: **SELECT, FROM, WHERE, CREATE, ...**)
- Lowercase letters are used for all other words.
- Vertical bar indicates that you can choose between several components. (Ex: DISTINCT|ALL)
- Brackets indicate a required element. (Ex: {*****|columnName})
- Square brackets indicate an optional element. (Ex: [WHERE condition])
- Ellipsis indicates optional repetition of an item zero or more times. (Ex: FROM table [,...])

SQL commands

An SQL command contains:

- Reserved words
- User-defined words

It must end with ';'.

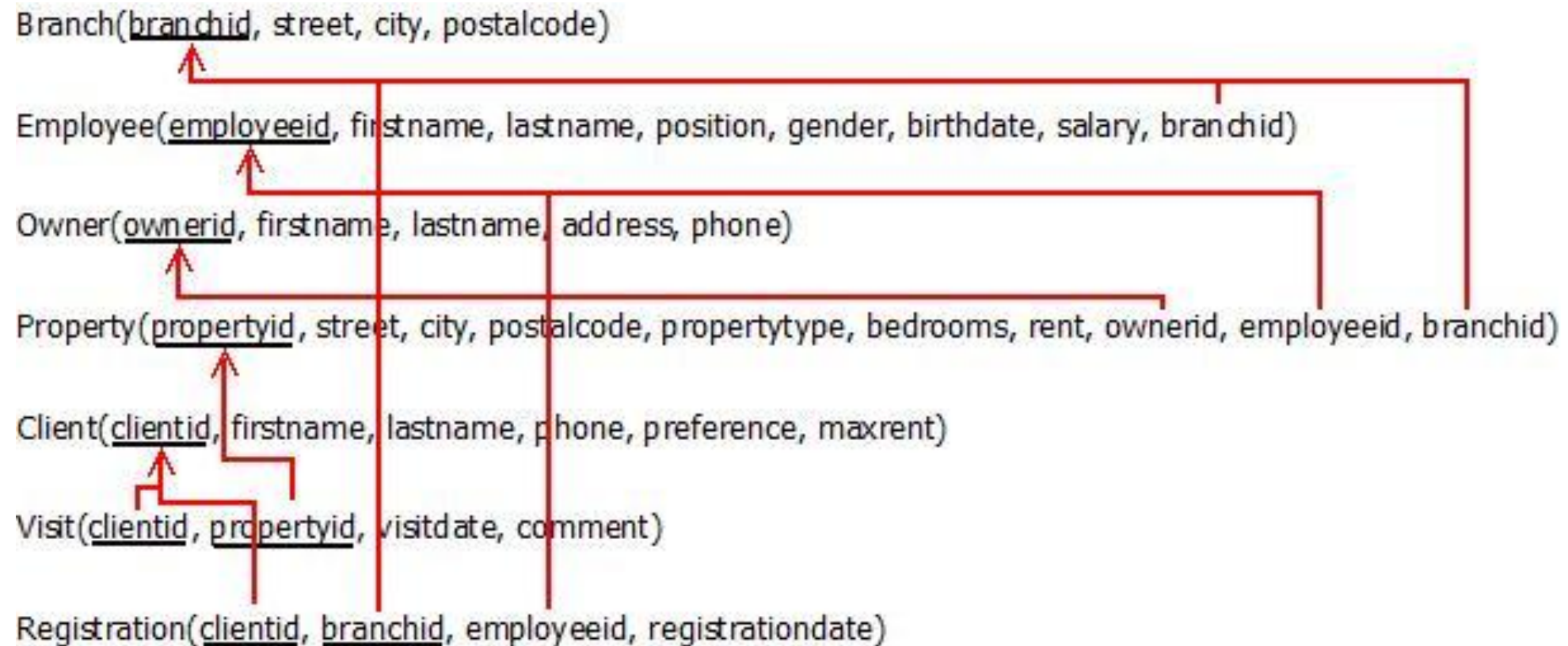
Data in tables is case-sensitive.

- 'Maria' is not the same as 'MARIA'.

-
- Fixed part of the language
 - They must be written exactly as expected
 - They are not case sensitive.
 - SELECT, Select and select are all valid.
 - To represent database objects (tables, columns, views, etc.)
 - They can contain uppercase letters, lowercase letters, numbers, and underscores.
 - They cannot exceed 128 characters.
 - In some cases, the limit is lower.
 - They must start with a letter.
 - They cannot contain spaces.

Example database structre

The examples shown are based on the **propertymanagement** database and have the following structure:



Create and delete a database

■ Create a database

```
CREATE SCHEMA [name | AUTHORIZATION creatorId]
```

```
CREATE DATABASE [name | AUTHORIZATION creatorId]
```

```
CREATE SCHEMA propertymanagement;
```

```
CREATE DATABASE propertymanagement;
```

■ Delete a database

```
DROP SCHEMA [RESTRICT | CASCADE]
```

```
DROP DATABASE [RESTRICT | CASCADE]
```

```
DROP SCHEMA propertymanagement;
```

```
DROP DATABASE propertymanagement;
```


Create and delete a table

```
CREATE TABLE name
{(column dataType [NOT NULL] [UNIQUE]
[DEFAULT option] [CHECK (condition)] [, ... ]}
[PRIMARY KEY (listOfColumns),]
{[UNIQUE (listOfColumns)] [, ... ]}
{[FOREIGN KEY (listForeigKeys)
REFERENCES foreignKeyTable [(listKeys)]
[MATCH {PARTIAL | FULL}
[ON UPDATE action]
[ON DELETE action]] [, ... ]}
{[CHECK (condition)] [, ... ]})
```

```
DROP TABLE tabela [RESTRICT | CASCADE]
```

```
CREATE TABLE branch (
    branchid CHAR(4) PRIMARY KEY,
    street VARCHAR(30),
    city VARCHAR(30),
    postalcode VARCHAR(10)
);
```

```
DROP TABLE branch;
```

Data Types

Type	Declaration
Boolean	BOOLEAN
Character	CHAR – quando o tamanho é fixo VARCHAR – quando o tamanho é variável
Bit	BIT BIT VARYING
Numeric (exact)	NUMERIC DECIMALL INTEGER SMALLINT
Numeric (approximate)	FLOAT REAL DOUBLE
Date / Time	DATE TIME TIMESTAMP
Interval	INTERVAL
Large objects	CHARACTER LARGE OBJECT BINARY LARGE OBJECT

Modify tables

ALTER TABLE name

[ADD [COLUMN] column dataType [NOT NULL] [UNIQUE]

[DEFAULT default] [CHECK (condition)]]

[DROP [COLUMN] column [RESTRICT | CASCADE]]

[ADD [CONSTRAINT [constraint]] constraintDefinition]

[DROP CONSTRAINT constraint [RESTRICT | CASCADE]]

[ALTER [COLUMN] SET DEFAULT default]

[ALTER [COLUMN] DROP DEFAULT]

Add / delete column

Add / delete constraint

Add / delete column default value

Modify the employee table by removing the default 'Assistant' from the job position and setting the default for the gender column to 'F'.

```
ALTER TABLE employee
    ALTER position DROP DEFAULT;
ALTER TABLE employee
    ALTER gender SET DEFAULT 'F';
```

Modify the property table by removing the restriction that employees cannot manage more than 100 residences. Modify the client table by adding a new column representing the preference regarding the number of bedrooms.

```
ALTER TABLE property
    DROP CONSTRAINT lessThan100;
ALTER TABLE client
    ADD prefNBedRooms dQuartos;
```

Create and delete indexes

- Create an index

```
CREATE [UNIQUE] INDEX name  
ON table(column [ASC|DESC][,...])
```

```
CREATE INDEX idx_city  
ON branch(city(10));
```

- Delete an index

```
DROP INDEX name
```

```
DROP INDEX idx_city;
```

SELECT

```
SELECT      [DISTINCT | ALL] {*} | [column [AS newName]] [, . . . ]}  
FROM        table [alias] [, . . . ]  
[WHERE      condition]  
[GROUP BY   listColumns] [HAVING condition]  
[ORDER BY   listColumns]
```

SELECT: Specifies the columns that should appear in the result

FROM: Specifies the table(s) to be used

WHERE: Filters rows according to a condition

GROUP BY: Forms groups of rows with the same value in a column

HAVING: Filters groups according to a condition

ORDER BY: Specifies the order of the result

Select all rows and all columns.

List all employee details

```
SELECT *  
FROM employee;
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SA9	Maria	Marques	Assistant	F	1970-02-19	9000	B007
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG37	Ana	Santos	Assistant	F	1960-11-10	12000	B003
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005
SL41	Julia	Borges	Assistant	F	1965-05-13	9000	B005

```
SELECT employeeid, firstname, lastname, position, gender, birthdate, salary, branchid  
FROM employee;
```

Select specific columns, all rows

Generate a list of employee salaries, showing only the employee number, first and last name, and salary.

```
SELECT employeeid, firstname, lastname, salary  
FROM employee;
```

employeeid	firstname	lastname	salary
SA9	Maria	Marques	9000
SG14	David	Ferreira	18000
SG37	Ana	Santos	12000
SG5	Susana	Silva	24000
SL21	Joao	Alves	30000
SL41	Julia	Borges	9000

DISTINCT

List the numbers of all properties that have been visited.

```
SELECT propertyid  
FROM visit;
```

propertyid

PA14

PA14

PG36

PG4

PG4



```
SELECT DISTINCT propertyid  
FROM visit;
```

propertyid

PA14

PG36

PG4



Calculated fields

Generate a list of employees' monthly salaries, showing the employee number, first and last name, and salary.

```
SELECT employeeid, firstname, lastname, salary/12  
FROM employee;
```

employeeid	firstname	lastname	salary/12
SA9	Maria	Marques	750.0000
SG14	David	Ferreira	1500.0000
SG37	Ana	Santos	1000.0000
SG5	Susana	Silva	2000.0000
SL21	Joao	Alves	2500.0000
SL41	Julia	Borges	750.0000

```
SELECT employeeid, firstname, lastname, salary/12 AS monthlySalary  
FROM employee;
```

employeeid	firstname	lastname	monthlySalary
SA9	Maria	Marques	750.0000
SG14	David	Ferreira	1500.0000
SG37	Ana	Santos	1000.0000
SG5	Susana	Silva	2000.0000
SL21	Joao	Alves	2500.0000
SL41	Julia	Borges	750.0000

Filtering rows (WHERE)

Basic conditions:

- **Comparison:** compare the value of one expression with that of another expression
 - =, <, >, !=, <=, >=
 - AND and OR for more complex expressions
 - Expression evaluated from left to right
 - Sub-expressions in parentheses evaluated first
 - NOT evaluated before AND and OR
 - AND evaluated before OR
 - Use of parentheses is recommended to avoid ambiguities
- **Interval:** check if the value of an expression is within a given range
- **Member:** check if the value of an expression belongs to a set of values
- **Pattern:** check if the value of an expression matches a given pattern
- **Null:** check if the value of an expression is NULL

Comparison

List addresses of all branches in Porto or Braga

```
SELECT *  
FROM branch  
WHERE city='Porto' OR city='Braga';
```

branchid	street	city	postalcode
B003	Central St. 34	Braga	1122
B007	Av. Aliados 2345	Porto	4321

Interval (BETWEEN / NOT BETWEEN)

List all employees with salaries between 20000 and 30000.

```
SELECT *  
FROM employee  
WHERE salary BETWEEN 20000 AND 30000;
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005

```
SELECT *  
FROM employee  
WHERE salary>=20000 AND salary<=30000;
```

Member (IN / NOT IN)

List all managers and supervisors.

```
SELECT *  
FROM employee  
WHERE position IN ('Supervisor','Manager');
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005

```
SELECT *  
FROM employee  
WHERE position='Supervisor'  
       OR position='Manager';
```

Patterns (% and _)

% represents any sequence of characters

_ represents a character

LIKE 'H%': the first character must be an H, but the rest can be anything

LIKE 'H___': the string must have exactly 4 characters and the first must be an H

LIKE '%e': any sequence of characters that ends in an e

LIKE '%Braga%': a sequence of characters of any size that contains 'Braga'

NOT LIKE 'H%': the first character cannot be an H

If the string can contain the character % or _, we must use an escape character.

Example: **LIKE '15#%' ESCAPE '#'** to find the string '15%'

Patterns

List all property owners who live in Braga.

```
SELECT *  
FROM owner  
WHERE address LIKE '%Braga%';
```

ownerid	firstname	lastname	address	phone
CO40	Telma	Silva	Right St. 123 Braga	253986758
CO87	Carolina	Marques	Middle St. 54 Braga	253985746
CO93	Antonio	Magalhaes	Main St. 987 Braga	255098345

NULL

List details of visits to the PG4 property where the client did not leave a comment.

```
SELECT *  
FROM visit  
WHERE propertyid='PG4'  
      AND comment IS NULL;
```

clientid	propertyid	visitdate	comment
CR56	PG4	2004-05-26	NULL

Sorting (one column)

List the employees in descending order of salary.

```
SELECT *  
FROM employee  
ORDER BY salary DESC;
```

employeeid	firstname	lastname	position	gender	birthdate	salary ▼ 1	branchid
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG37	Ana	Santos	Assistant	F	1960-11-10	12000	B003
SA9	Maria	Marques	Assistant	F	1970-02-19	9000	B007
SL41	Julia	Borges	Assistant	F	1965-05-13	9000	B005

Sorting (several columns)

List the properties sorted by type and rental price in ascending order.

```
SELECT *  
FROM property  
ORDER BY propertytype, rent ASC;
```

propertyid	street	city	postalcode	propertytype ▲ 1	bedrooms	rent ▲ 2	ownerid	employeeid	branchid
PG4	Right St. 54	Braga	1122	Apartment	3	350	CO40	NULL	B003
PG36	Central Ave. 32	Braga	1122	Apartment	3	375	CO93	SG37	B003
PL94	Main St. 42	Felgueiras	123	Apartment	4	400	CO87	SL41	B005
PG16	Sameiro St. 87	Braga	1122	Apartment	4	450	CO93	SG14	B003
PG21	Good Jesus St. 32	Braga	1122	House	5	600	CO87	SG37	B003
PA14	Av. Boavista 12	Porto	4321	House	6	650	CO46	SA9	B007

Aggregation functions

COUNT, SUM, AVG, MIN, MAX

- They operate on only one column.
- COUNT, MIN, and MAX can be used with numeric and non-numeric fields.
- SUM and AVG can only be used with numeric fields.
- Each function eliminates NULLs and operates only on non-null values.
 - COUNT(*) counts all rows in the table, including nulls.
- DISTINCT has no effect on MIN and MAX, but may affect SUM and AVG.
 - A query can only have one DISTINCT.
- Aggregation functions can only be used in SELECT or HAVING lists.

COUNT(*)

How many properties have a rent higher than 350?

```
SELECT COUNT(*) AS rentAbove350  
FROM property  
WHERE rent>350;
```

rentAbove350
5

COUNT(DISTINCT)

How many different properties were visited in May 2004?

```
SELECT COUNT(DISTINCT propertyid) AS visitsMay2004  
FROM visit  
WHERE visitdate BETWEEN '2004-05-01' AND '2004-05-31';
```

visitsMay2004

2

COUNT and SUM

What is the number of managers and the sum of their salaries?

```
SELECT COUNT(employeeid) AS nManagers, SUM(salary) AS sumSalaries  
FROM employee  
WHERE position='Manager';
```

nManagers	sumSalaries
2	54000

MIN, MAX, AVG

What are the minimum, maximum, and average salaries for employees?

```
SELECT MIN(salary) AS minimum, MAX(salary) AS maximum, AVG(salary) AS average  
FROM employee;
```

minimum	maximum	average
9000	30000	17000.0000

Grouping (GROUP BY)

How many employees work in each branch and what is the sum of their salaries?

```
SELECT branchid, COUNT(employeeid) AS nrEmployees, SUM(salary) AS sumSalaries  
FROM employee  
GROUP BY branchid;
```

branchid	nrEmployees	sumSalaries
B003	3	54000
B005	2	39000
B007	1	9000

Group restrictions (HAVING)

Used with GROUP BY to apply restrictions to the groups that appear in the results.

Syntax similar to WHERE, but WHERE filters rows, while HAVING filters groups.

Columns that appear in HAVING must also appear in GROUP BY.

HAVING

For each branch with more than one employee, find the number of employees working in each branch and the sum of their salaries.

```
SELECT branchid, COUNT(employeeid) AS nrEmployees, SUM(salary) AS somaSalaries  
FROM employee  
GROUP BY branchid  
HAVING COUNT(employeeid)>1;
```

branchid	nrEmployees	somaSalaries
B003	3	54000
B005	2	39000

Updating the database: INSERT

```
INSERT INTO table [(listOfColumns)]  
VALUES (listOfValues)
```

Insert a new row into the employee table, providing data for all columns.

```
INSERT INTO employee  
VALUES('SG16','Andre','Brito','Assistente',  
'M','1957-5-25',8300,'B003');
```

Insert a new row into the employee table providing data for employeeid, firstname, lastname, position, salary, and branchid

```
INSERT INTO employee  
(employeeid, firstname, lastname, position, salary, branchid)  
VALUES('SG16','Andre','Brito','Assitente',8300,'B003')
```

```
INSERT INTO employee  
VALUES('SG16','Andre','Brito','Assitente',  
NULL, NULL, 8300, 'B003')
```

```
INSERT INTO table [(listOfColumns)]  
  
SELECT ...
```

If we have a table called "GeneralEmployees" and we want to insert only the name of each employee and the branch where they work,

```
INSERT INTO GeneralEmployees  
SELECT firstname, lastname, branchid  
FROM employee;
```

The query inside the SELECT statement can be any query, as long as it has the same columns as the table we want to populate.

Updating the database: UPDATE

```
UPDATE table  
SET coluna1 = valor1 [, coluna2 = valor2, ...]  
[WHERE condicao]
```

Update all rows: Give all employees a 3% raise

```
UPDATE employee  
SET salary = salary*1.03;
```

Update specific lines: give all managers a 5% raise.

```
UPDATE employee  
SET salary = salary*1.05  
WHERE position = 'Manager';
```

Update multiple columns: promote employee 'SG14' to the position of manager and increase their salary to 18000

```
UPDATE employee  
SET position = 'Manager', salary=18000  
WHERE employeeid = 'SG14';
```

Updating the database: DELETE

```
DELETE FROM table  
[WHERE condition]
```

Delete all rows: delete all rows from the Visits table.

```
DELETE FROM visit
```

Remove specific rows: remove all visits related to the property 'PG4'

```
DELETE FROM visit  
WHERE propertyid = 'PG4';
```

Deleting foreign keys

CASCADE: When you delete the value in the main table, you delete all rows in the other tables that have that value in the foreign key.

SET NULL: When you delete the value in the main table, it is replaced with NULL in the dependent tables.

SET DEFAULT: When the value is deleted in the main table, it is replaced with the default value in the dependent tables, if any.

NO ACTION: The delete operation is rejected.

Example:

```
FOREIGN KEY (employeeid) REFERENCES  
employee(employeeid)  
ON DELETE SET NULL
```

Subqueries

- Using a SELECT statement inside another. Representation in parentheses.
 - The result of the "inner" SELECT statement is used to calculate the "outer" SELECT statement.
 - It can be used within WHERE and HAVING clauses.
 - They can be of the following types:
 - Scalar: returned a single value
 - Row: returned a row
 - Table: returned a table
-

Subqueries with comparison

List the employees who work at the **branch located at 'Central St. 34'**

```
SELECT *  
FROM employee  
WHERE branchid = (  
    SELECT branchid  
    FROM branch  
    WHERE street = 'Central St. 34'  
);
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG37	Ana	Santos	Assistant	F	1960-11-10	12000	B003
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003

Subquery and aggregation function

List the employees whose salary is **higher than the average salary** and show by **how much it is higher than the average**.

```
SELECT  
employeeid, firstname, lastname, position, salary,  
    salary-(SELECT AVG(salary) FROM employee) AS difference  
FROM employee  
WHERE salary > (  
    SELECT AVG(salary)  
    FROM employee  
);
```

employeeid	firstname	lastname	position	salary	difference
SG14	David	Ferreira	Supervisor	18000	1000.0000
SG5	Susana	Silva	Manager	24000	7000.0000
SL21	Joao	Alves	Manager	30000	13000.0000

Subqueries (IN)

List the properties managed by employees who work at the branch on 'Central St. 34'

```
SELECT *  
FROM property  
WHERE employeeid IN (  
    SELECT employeeid  
    FROM employee  
    WHERE branchid = (  
        SELECT branchid  
        FROM branch  
        WHERE street='Central St. 34'  
    )  
);
```

propertyid	street	city	postalcode	propertytype	bedrooms	rent	ownerid	employeeid	branchid
PG16	Sameiro St. 87	Braga	1122	Apartment	4	450	CO93	SG14	B003
PG21	Good Jesus St. 32	Braga	1122	House	5	600	CO87	SG37	B003
PG36	Central Ave. 32	Braga	1122	Apartment	3	375	CO93	SG37	B003

ANY / SOME

List all employees whose salary is higher than the salary **of at least one** employee at branch B003.

```
SELECT *  
FROM employee  
WHERE salary >  
SOME (  
    SELECT salary  
    FROM employee  
    WHERE branchid='B003'  
);
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005

ALL

List the employees whose salary is higher than the salaries **of all** employees in branch B003.

```
SELECT *  
FROM employee  
WHERE salary >  
ALL (  
    SELECT salary  
    FROM employee  
    WHERE branchid='B003'  
);
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SL21	Joao	Alves	Manager	M	1945-10-01	30000	B005

Queries with multiple tables: simple JOIN

List the IDs and names of all customers who visited a property, the property visited, and the comments.

clientid	firstname	lastname	clientid	propertyid	visitdate	comment
CR56	Adelina	Santos	CR56	PA14	2004-05-24	too small
CR62	Maria	Pereira	CR56	PG36	2004-04-28	NULL
CR74	Miguel	Silva	CR56	PG4	2004-05-26	NULL
CR76	Joao	Alves	CR62	PA14	2004-05-14	no dining room
			CR76	PG4	2004-04-20	too far away

```
SELECT c.clientid, firstname, lastname,
       propertyid, comment
FROM client c, visit v
WHERE c.clientid = v.clientid;
```

```
SELECT c.clientid, firstname, lastname,
       propertyid, comment
FROM client c JOIN visit v
      ON c.clientid = v.clientid;
```

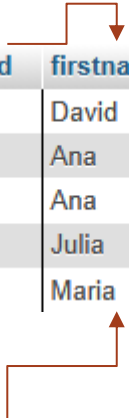
clientid	firstname	lastname	propertyid	comment
CR56	Adelina	Santos	PA14	too small
CR56	Adelina	Santos	PG36	NULL
CR56	Adelina	Santos	PG4	NULL
CR62	Maria	Pereira	PA14	no dining room
CR76	Joao	Alves	PG4	too far away

Sorting a JOIN

For each branch, list the numbers and names of the employees who manage properties and managed properties.

```
SELECT e.branchid, e.employeeid, firstname, lastname, propertyid
FROM employee e, property p
WHERE e.employeeid = p.employeeid
ORDER BY e.branchid, e.employeeid, propertyid;
```

```
SELECT e.branchid, e.employeeid, firstname, lastname, propertyid
FROM employee e JOIN property p
      ON e.employeeid = p.employeeid
ORDER BY e.branchid, e.employeeid, propertyid;
```



branchid ▲ 1	employeeid	firstname	lastname	propertyid ▲ 3
B003	SG14	David	Ferreira	PG16
B003	SG37	Ana	Santos	PG21
B003	SG37	Ana	Santos	PG36
B005	SL41	Julia	Borges	PL94
B007	SA9	Maria	Marques	PA14

JOIN three tables

For each branch, list the numbers and names of the employees who manage properties, including the branch's city and the properties the employee manages.

branchid	city	employeeid	firstname	lastname	branchid	propertyid	employeeid
B002	Felgueiras	SA9	Maria	Marques	B007	PG4	NULL
B003	Braga	SG14	David	Ferreira	B003	PA14	SA9
B004	Lousada	SG37	Ana	Santos	B003	PG16	SG14
B005	Felgueiras	SG5	Susana	Silva	B003	PG21	SG37
B007	Porto	SL21	Joao	Alves	B005	PG36	SG37
		SL41	Julia	Borges	B005	PL94	SL41

branchid	city	employeeid	firstname	lastname	propertyid
B003	Braga	SG14	David	Ferreira	PG16
B003	Braga	SG37	Ana	Santos	PG21
B003	Braga	SG37	Ana	Santos	PG36
B005	Felgueiras	SL41	Julia	Borges	PL94
B007	Porto	SA9	Maria	Marques	PA14

```
SELECT b.branchid, b.city, e.employeeid, firstname, lastname, propertyid
FROM branch b, employee e, property p
WHERE b.branchid = e.branchid
      AND e.employeeid = p.employeeid;
```

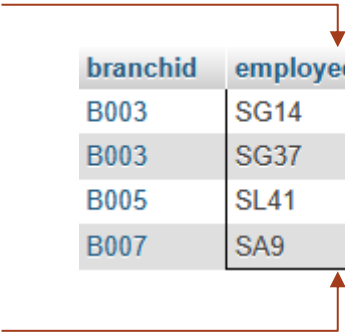
```
SELECT b.branchid, b.city, e.employeeid, firstname, lastname, propertyid
FROM branch b JOIN employee e ON b.branchid = e.branchid
      JOIN property p ON e.employeeid = p.employeeid;
```

Group by multiple columns

Find the number of properties managed by each employee.

```
SELECT e.branchid, e.employeeid, COUNT(*) AS nProperties
FROM employee e, property p
WHERE e.employeeid = p.employeeid
GROUP BY e.branchid, e.employeeid;
```

```
SELECT e.branchid, e.employeeid, COUNT(*) AS nProperties
FROM employee e JOIN property p
      ON e.employeeid = p.employeeid
GROUP BY e.branchid, e.employeeid;
```



branchid	employeeid	nProperties
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

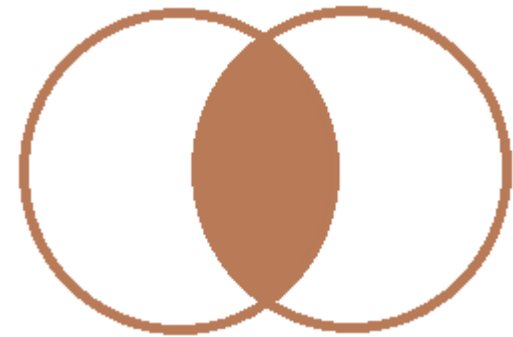
INNER JOIN

left_table

l1	l2
1	A
2	B
3	C

right_table

r1	r2
2	X
3	Y
4	Z



```
SELECT *  
FROM left_table JOIN right_table  
      ON l1=r1;
```

l1	l2	r1	r2
2	B	2	X
3	C	3	Y

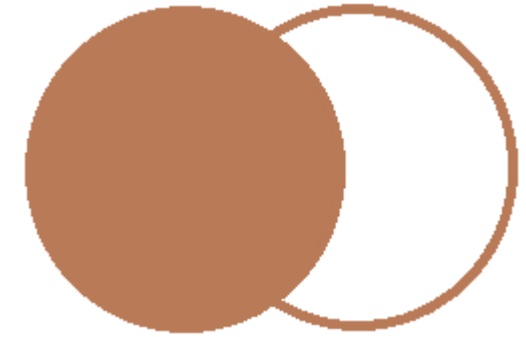
LEFT OUTER JOIN

left_table

l1	l2
1	A
2	B
3	C

right_table

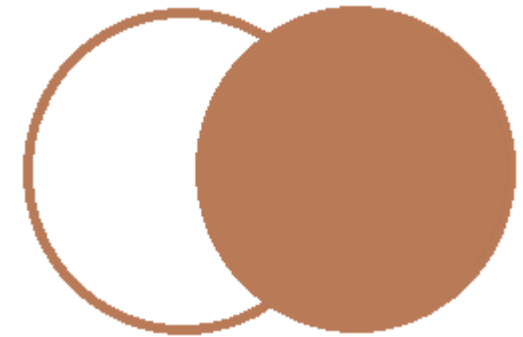
r1	r2
2	X
3	Y
4	Z



```
SELECT *
FROM left_table LEFT JOIN right_table
      ON l1=r1;
```

l1	l2	r1	r2
2	B	2	X
3	C	3	Y
1	A	NULL	NULL

RIGHT OUTER JOIN



left_table

l1	l2
1	A
2	B
3	C

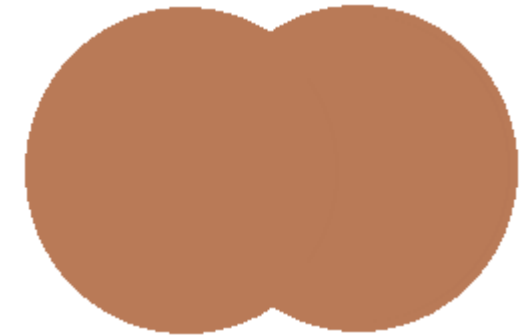
right_table

r1	r2
2	X
3	Y
4	Z

```
SELECT * FROM
left_table RIGHT JOIN right_table
ON l1=r1;
```

l1	l2	r1	r2
2	B	2	X
3	C	3	Y
NULL	NULL	4	Z

FULL OUTER JOIN



left_table		right_table	
l1	l2	r1	r2
1	A	2	X
2	B	3	Y
3	C	4	Z

```

SELECT *
FROM left_table LEFT JOIN right_table
      ON l1=r1
UNION
SELECT *
FROM left_table RIGHT JOIN right_table
      ON l1=r1;

```

l1	l2	r1	r2
2	B	2	X
3	C	3	Y
1	A	NULL	NULL
NULL	NULL	4	Z

EXISTS

Find all the employees who work at the Felgueiras branch.

```
SELECT employeeid, firstname, lastname, position
FROM employee e
WHERE EXISTS (
    SELECT *
    FROM branch b
    WHERE e.branchid = b.branchid
        AND city='Felgueiras'
);
```

employeeid	firstname	lastname	position
SL21	Joao	Alves	Manager
SL41	Julia	Borges	Assistant

Combine table results

$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

- Union: elements that belong to A or B
 - $A \cup B = \{1, 2, 3, 4, 5, 6\}$
- Intersection: elements that belong to both A and B
 - $A \cap B = \{3, 4\}$
- Difference: elements that belong to A but not to B
 - $A - B = \{1, 2\}$

The tables must have the same number of columns and the data types must be the same.

Syntax: `Operator [ALL][CORRESPONDING [BY {column1[,...]}]]` Operator = UNION, INTERSECT, EXCEPT

UNION

Create a list of all cities where there is a branch **or** property.

```
(  
SELECT city  
FROM branch  
WHERE city IS NOT NULL  
)  
UNION  
(  
SELECT city  
FROM property  
WHERE city IS NOT NULL  
);
```

city
Felgueiras
Braga
Lousada
Porto

INTERSECT

Create a list of all cities where there is a branch **and** a property.

```
SELECT DISTINCT city
FROM branch
WHERE city IN (
    SELECT city
    FROM property
);
```

city
Felgueiras
Braga
Porto

EXCEPT

Create a list of all cities where there is a branch **but not** a property.

```
SELECT DISTINCT city
FROM branch
WHERE city NOT IN (
    SELECT city
    FROM property
);
```

city
Lousada

VIEW

Dynamic results of one or more relational operations on basic relations to produce new relations. A view is a virtual relationship that does not necessarily exist in the database, but can be produced by a specific user's request.

```
CREATE VIEW name [(newColumn [, ...])]  
AS query [WITH [CASCADED | LOCAL] CHECK OPTION]
```

```
DROP VIEW name [RESTRICT | CASCADE]
```

Advantages and Disadvantages of VIEW

Advantages

- Data independence
- Changes in tables are reflected in views
- Greater security
- Reduced complexity
- Convenience
- Customization
- Data integrity

Disadvantages

- Restrictions on updates
- Structure restrictions
- Performance

Horizontal VIEW

Create a view so that the manager of branch B003 can see only the details of the employees who work in their branch.

```
CREATE VIEW branchb003 AS
  SELECT *
  FROM employee
  WHERE branchid = 'B003';
```

```
SELECT * FROM branchb003;
```

employeeid	firstname	lastname	position	gender	birthdate	salary	branchid
SG14	David	Ferreira	Supervisor	M	1958-03-24	18000	B003
SG37	Ana	Santos	Assistant	F	1960-11-10	12000	B003
SG5	Susana	Silva	Manager	F	1940-06-03	24000	B003

Vertical VIEW

Create a detailed view of branch B003 employees without displaying salary information.

```
CREATE VIEW branchb003_less AS  
  SELECT employeeid, firstname, lastname, position, gender  
  FROM employee  
  WHERE branchid = 'B003';
```

```
SELECT * FROM branchb003_less;
```

employeeid	firstname	lastname	position	gender
SG14	David	Ferreira	Supervisor	M
SG37	Ana	Santos	Assistant	F
SG5	Susana	Silva	Manager	F

VIEW with GROUP and JOIN

Create a view of employees who manage properties, including the branch they work for, employee number, and number of properties they manage.

```
CREATE VIEW employee_property(branchid,employeeid,nProperties) AS
  SELECT e.branchid, e.employeeid, COUNT(*)
  FROM employee e, property p
  WHERE e.employeeid = p.employeeid
  GROUP BY e.branchid, e.employeeid;
```

```
SELECT* FROM employee_property;
```

branchid	employeeid	nProperties
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1